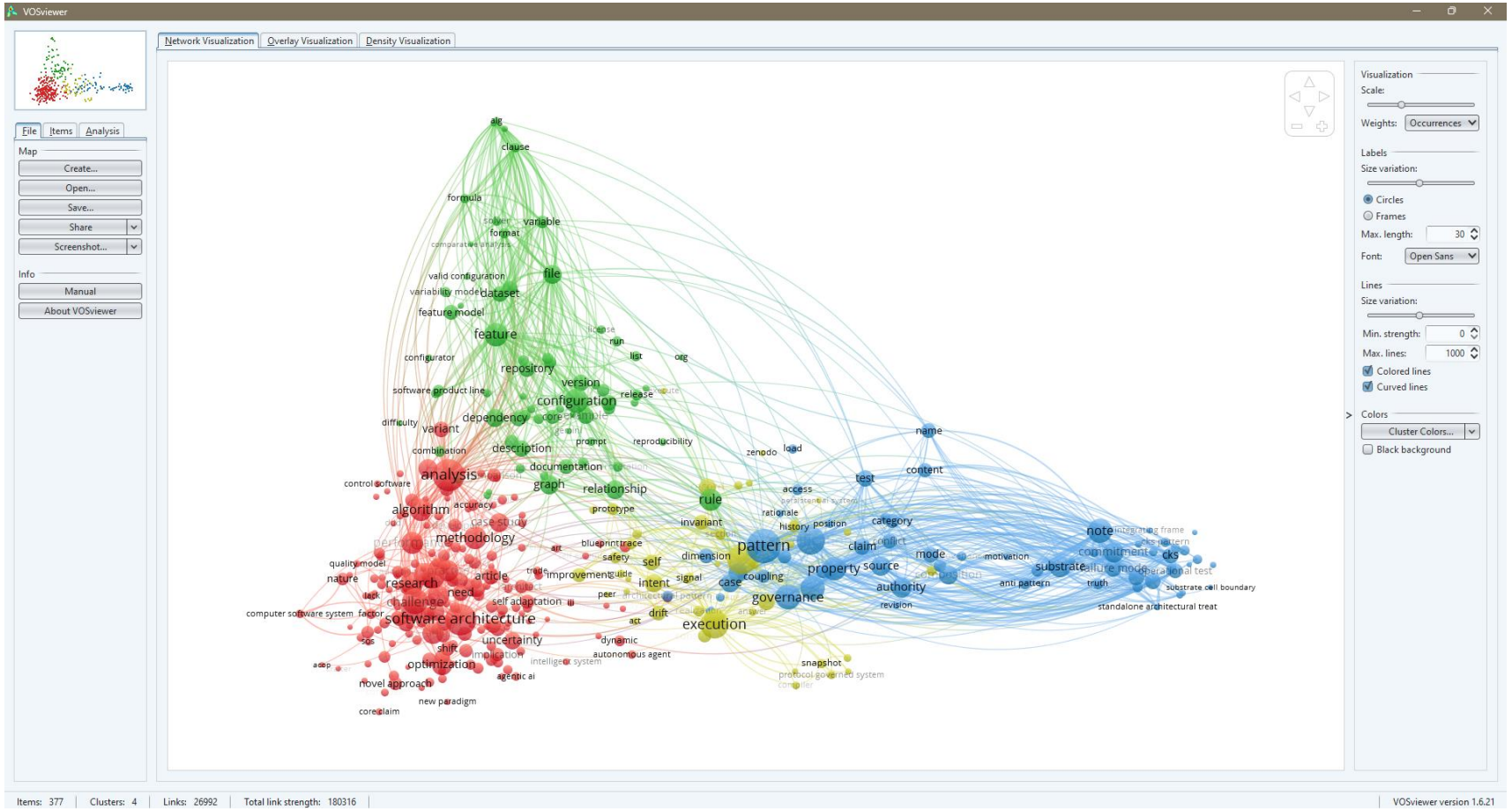
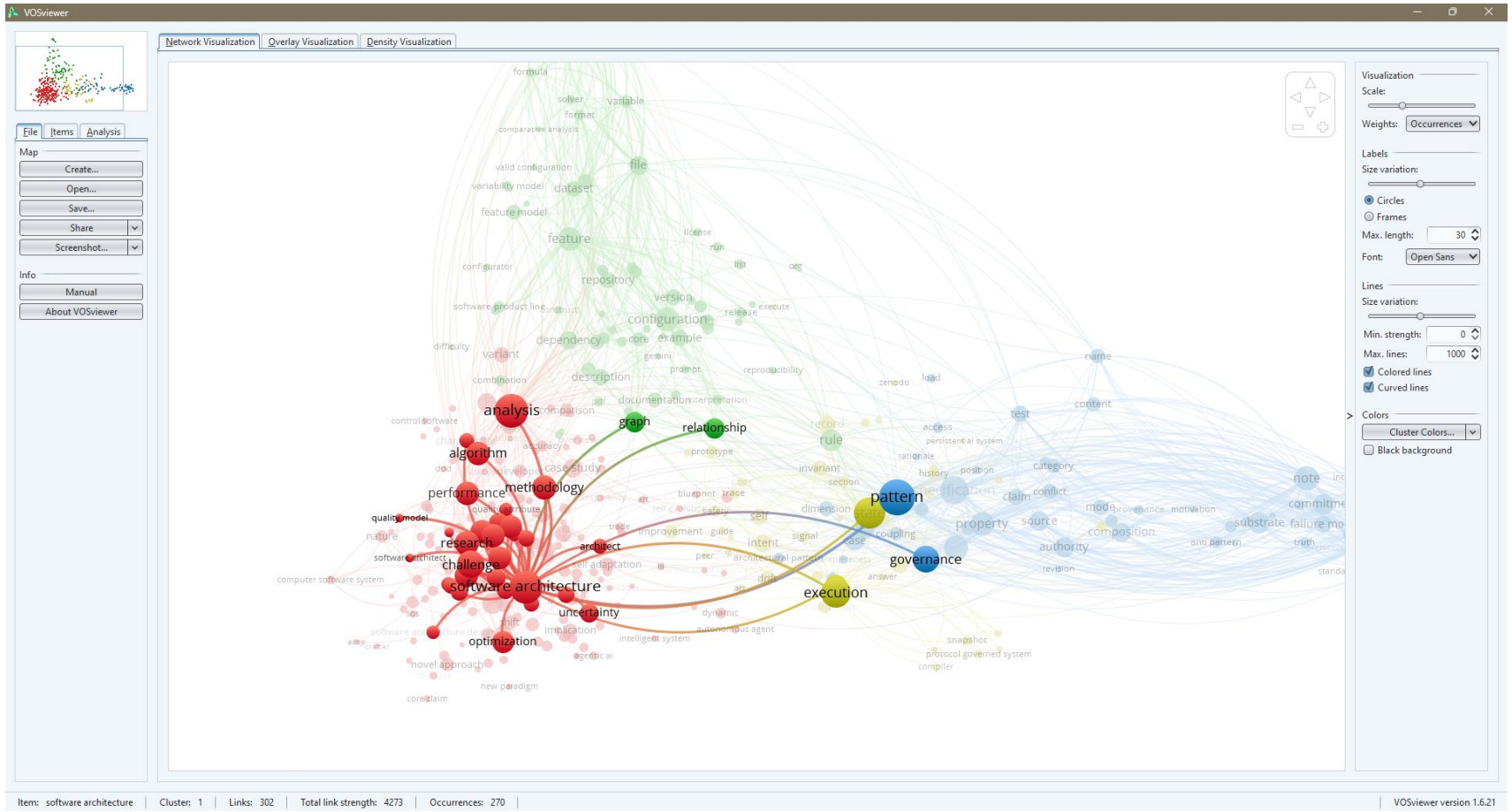
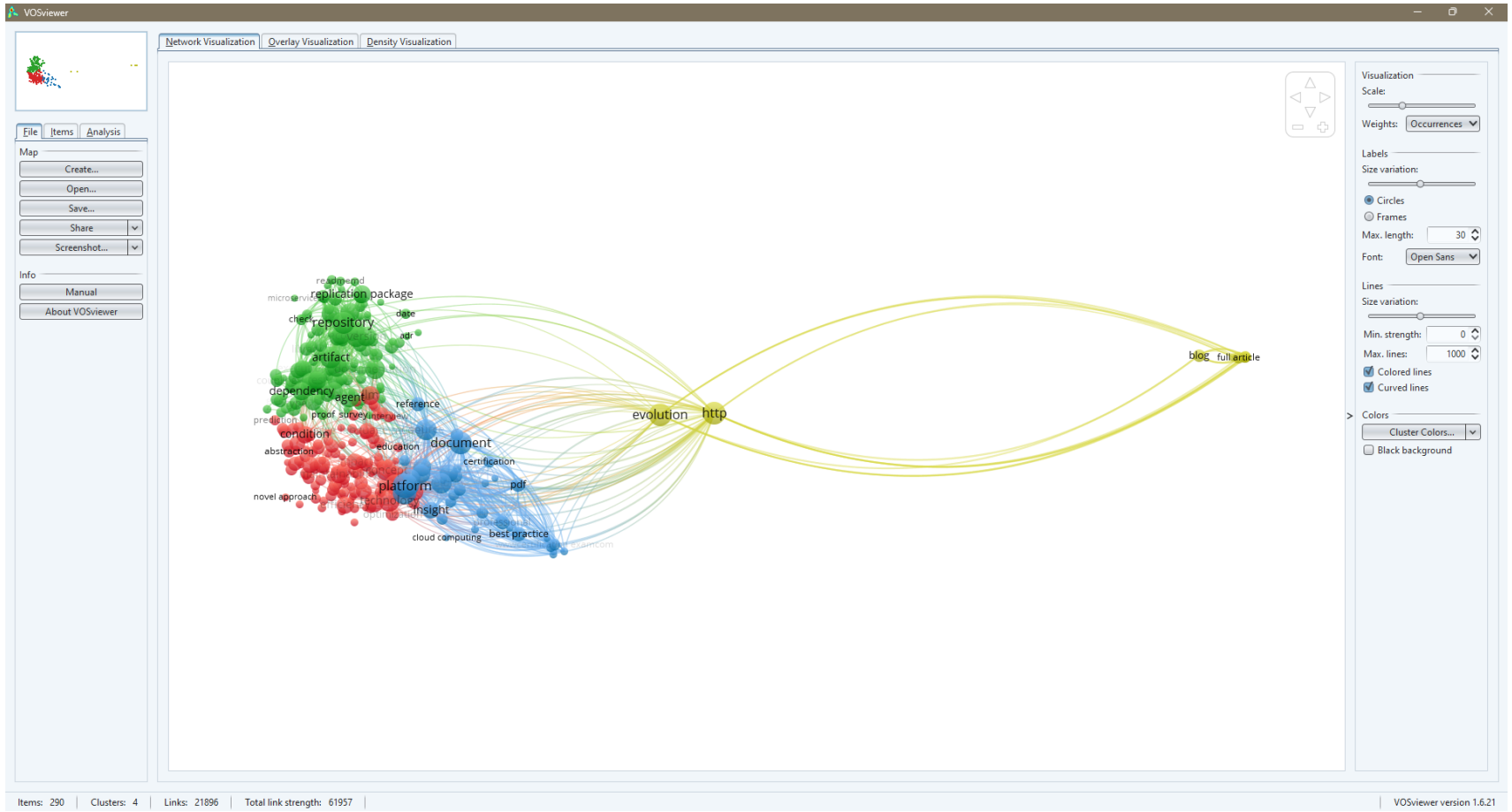
Основные темы в 2026 году в топе



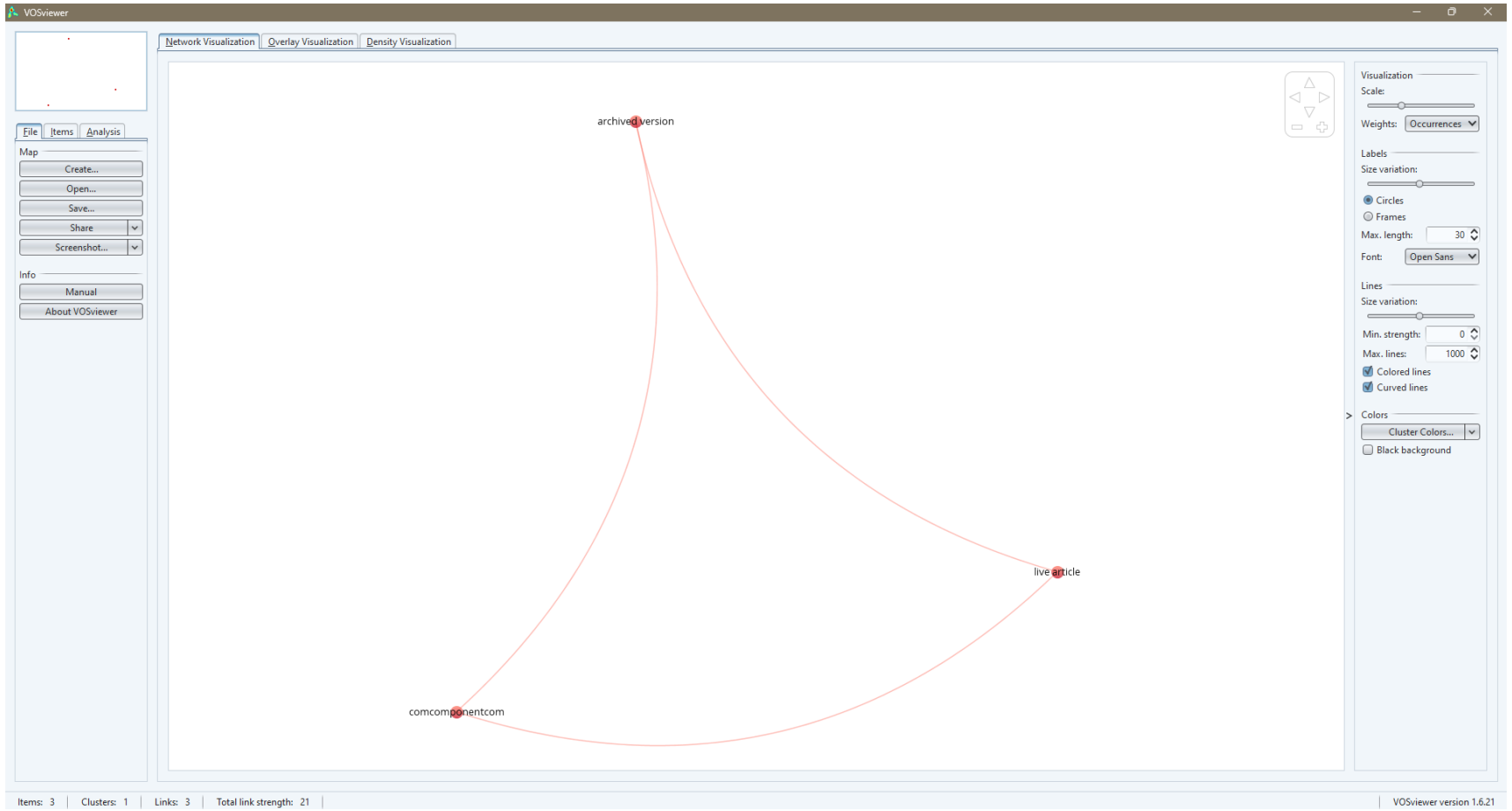
Software architecture



Разбитие темы на маленькие



Komura, Go · Zenodo



Как провести границу между модульным тестированием и интеграционным тестированием (архивировано 2026-10-31)

- 1. Главный вопрос статьи
- Как провести границу между модульными и интеграционными тестами: что именно выносить в быстрые модульные тесты, а что поднимать в интеграционные. Автор формулируют свой ответ одной фразой: модульный тест — это тест «решения» (логики), интеграционный — тест «соединения» (стыковки компонентов).
-

- Контекст исследования
- Проблема выбора слоя тестов — классическая тема тест-дизайна, на которую опираются два устоявшихся источника: рекомендации Microsoft Learn по модульным и интеграционным тестам в .NET/ASP.NET Core и «The Practical Test Pyramid» Мартина Фаулера (пирамида тестов: внизу — много быстрых модульных, наверху — мало медленных E2E). До настоящего исследования (то есть до этой статьи) общепринято, что модульные тесты должны быть быстрыми, изолированными и повторяемыми, а интеграционные — покрывать реальные сценарии инфраструктуры, но конкретный практический критерий «где резать границу» в призме реальной Windows-разработки (COM, права доступа, разрядность) обычно остаётся за кадром. Статья претендует на то, чтобы этот критерий сделать явным и применимым за один проход по чек-листу.

Конкретные вопросы, на которые отвечают авторы

- По какому признаку относить проверяемую функциональность к модульным или интеграционным тестам?
- Какие типичные ошибки допускают команды (например, Repository целиком в моках, попытка проверить фреймворк через модульные тесты контроллера, полный перебор входов в интеграционных тестах)?
- Что делать, если в модульном тесте слишком много моков — о чём это сигнализирует?
- Какая структура и соотношение тестов в проекте считаются рабочими на практике?
- Чем отличаются stub, mock и fake и как это различие использовать?

Результаты одним абзацем

- По данным авторов, границу между слоями тестов нужно проводить не по месту расположения кода, а по типу проверяемой неопределённости: всё, что завершается по смыслу без внешних ресурсов (расчёты, состояния, валидация, классификация ошибок), относится к модульным тестам, а четыре класса «стыковок» — формат (JSON/CSV/схемы БД/кодировки), разводка (DI, роутинг, model binding), окружение (права файлов, COM, разрядность, потоки) и время (таймауты, отмена, гонки) — к интеграционным. Рабочая конфигурация — три слоя: толстый пласт модульных тестов на перебор ветвлений, узкие интеграционные тесты по одной успешной и одной типичной ошибочной траектории на каждую границу, и небольшое число смоук/E2E для запуска и главных сценариев. Сигнал перекоса: 7 и более моков в одном модульном тесте или «всё зелёное, а прод падает» означает, что проверку стыковки ошибочно втиснули в модульный слой. При появлении бага тест добавляется на том слое, где этот баг воспроизводится минимальными затратами.